

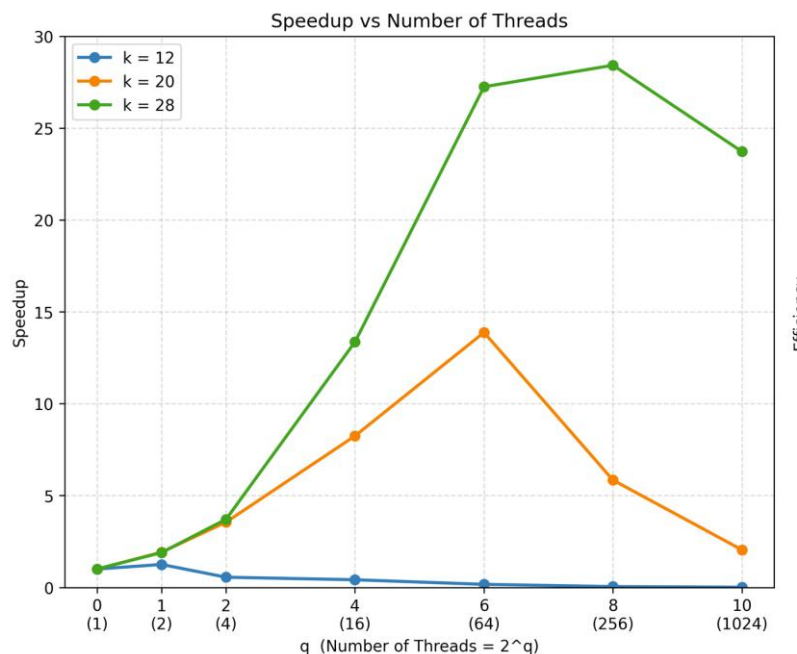
CSCE 735

HW 2: Parallel Merge Sort Using Threads

Name: Medha Venkata Subrahmanya Aditya Vellampalli

UIN: 237006700

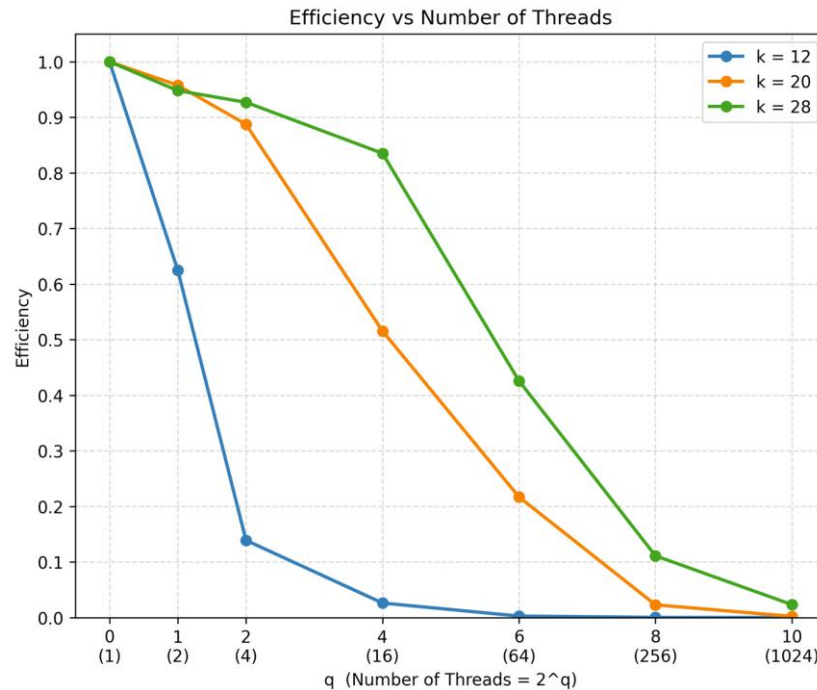
1. Uploaded `sort_list.c` in submission
2. (20 points) Plot speedup and efficiency for all combinations of k and q chosen from the following sets: $k = 12, 20, 28$; $q = 0, 1, 2, 4, 6, 8, 10$. Comment on how the results of your experiments align with or diverge from your understanding of the expected behavior of the parallelized code.



- The experimental results show that the performance of the parallelized code depends strongly on the problem size and the number of threads used.
- For ($k=12$), the speedup decreases as the number of threads increases. Because the input size is small, parallel overhead outweighs the computation, so more threads do not improve performance.
- For ($k=20$), the speedup increases up to 64 threads and then starts to decrease. This indicates that the workload benefits from parallel execution up to a point, after which thread management and synchronization overhead reduce performance gains.
- For ($k=28$), the largest input size shows the best parallel performance. The speedup increases up to 256 threads and then decreases slightly at 1024 threads. This behavior is expected because larger workloads can make better use of parallelism,

while very high thread counts lead to diminishing returns due to synchronization, scheduling, and memory overhead.

- Overall, the results are consistent with the expected behavior of the parallelized code. Larger problem sizes achieve better speedup, while excessive thread counts reduce efficiency.



3. (10 points) Your code should demonstrate speedup when sorting lists of appropriate sizes. Determine two values of k for which your code shows speedup as q is varied. Present the timing results for your code along with speedup and efficiency obtained to convince the reader that you have a well-designed parallel merge sort. You may use results from experiments in previous problems or identify new values k and q to illustrate how well your code has been parallelized.

- The values ($k=20$) and ($k=28$) were selected because both problem sizes show clear improvement in execution time as the number of threads increases.
- For ($k=20$), the execution time decreases from 0.1402 s with one thread to 0.0101 s with 64 threads, giving a maximum speedup of approximately 13.88. After 64 threads, the execution time begins to increase, showing that the parallel overhead becomes significant when too many threads are used.

q	Threads	Time (s)	Speedup	Efficiency
0	1	0.1402	1.00	1.000
1	2	0.0732	1.92	0.958
2	4	0.0395	3.55	0.887
4	16	0.0170	8.25	0.515
6	64	0.0101	13.88	0.217
8	256	0.0240	5.84	0.023
10	1024	0.0686	2.04	0.002

- For ($k=28$), the larger problem size shows stronger parallel performance. The execution time decreases from 49.8663 s with one thread to 1.7543 s with 256 threads, resulting in a maximum speedup of approximately 28.43. The efficiency remains relatively high for smaller thread counts, with about 92.7% efficiency at 4 threads and 83.5% at 16 threads. At 1024 threads, the execution time increases slightly, indicating diminishing returns from using additional threads.

q	Threads	Time (s)	Speedup	Efficiency
0	1	49.8663	1.00	1.000
1	2	26.3009	1.90	0.948
2	4	13.4490	3.71	0.927
4	16	3.7316	13.36	0.835
6	64	1.8297	27.25	0.426
8	256	1.7543	28.43	0.111
10	1024	2.1018	23.73	0.023

- These results show that the parallel merge sort provides good speedup for sufficiently large input sizes. The ($k=28$) case benefits more from parallelization because there is enough work available for each thread. The reduction in efficiency at higher thread counts is expected because synchronization, thread management, merging, and memory overhead become more significant as the number of threads increases.